

Exercice n°1 : (6 pts= 2.5 + 1+ 2.5)

Etant donné le programme suivant :

Debut

$X1 \leftarrow (a+b)*(c/d)$
 $X2 \leftarrow (a+b)*(c/d)-(e+f)/(g+h)$
 $X3 \leftarrow X2/((e+f)/(g+h))$

Fin.

A/ Donner son graphe de précédences de parallélisme maximal de manière intuitive.

B/ Ce graphe est-il proprement lié ?

C/ Exprimer ce graphe à l'aide des primitives Pargeni Parend et éventuellement les sémaphores.

Exercice n°2 : (7 pts=3,5+1.5+2)

Soit le système suivant composé de 3 processus :

Const $N = \dots$;

Var semaphore $nv1 := 1, nv2 := 1, np1 := 0, np2 := 0$;

$T1, T2$: Tableau $[0..N-1]$ de $Tart$; /* $Tart$ est un type de structure */

Processus $P()$;

Processus $P1()$;

Processus $P2()$;

Var $m : \dots$;

Var $m : \dots$;

Var $m : \dots$;

Debut

Debut

Debut

Repeter

Repeter

Repeter

Produire (m) ;

$P(np1)$;

$P(np2)$

$P(nv1)$; $P(nv2)$;

Prélever($T1, m$) ;

Prélever($T2, m$) ;

Déposer ($T1, m$) ;

$V(nv1)$;

$V(nv2)$

Déposer ($T2, m$) ;

Consommer (m)

Consommer (m)

$V(np1)$; $V(np2)$

Jusqu'à faux

Jusqu'à faux

Jusqu'à faux

Fin.

Fin.

Fin.

A/ Que fait cette solution ? Expliquer son fonctionnement.

B/ Quel est l'écart maximal entre les valeurs que peuvent prendre $np1$ et $np2$?

C/ Quels est l'inconvénient principal de cette solution?

Exercice n°3 : (7 pts=1+1+5)

On suppose un système qui contient N processus et une seule classe de ressource à M instances. Chaque processus peut demander k exemplaires et la priorité est donnée à celui qui demande le moins d'instances. On désire gérer l'allocation de cette classe de ressources à l'aide des moniteurs classiques.

A/ Donner la forme générale de chaque processus.

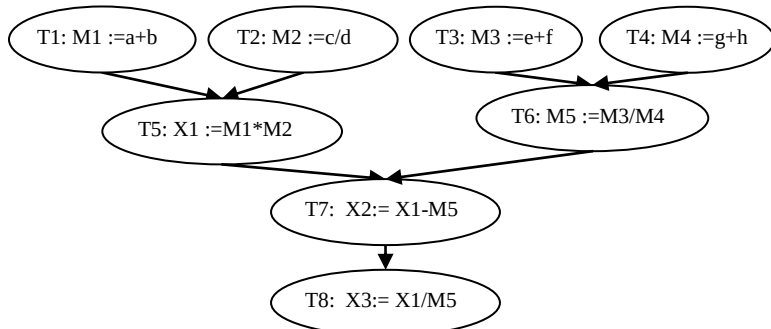
B/ Donner les structures de données principales

C/ Ecrire une solution correspondante.

Bon Courage

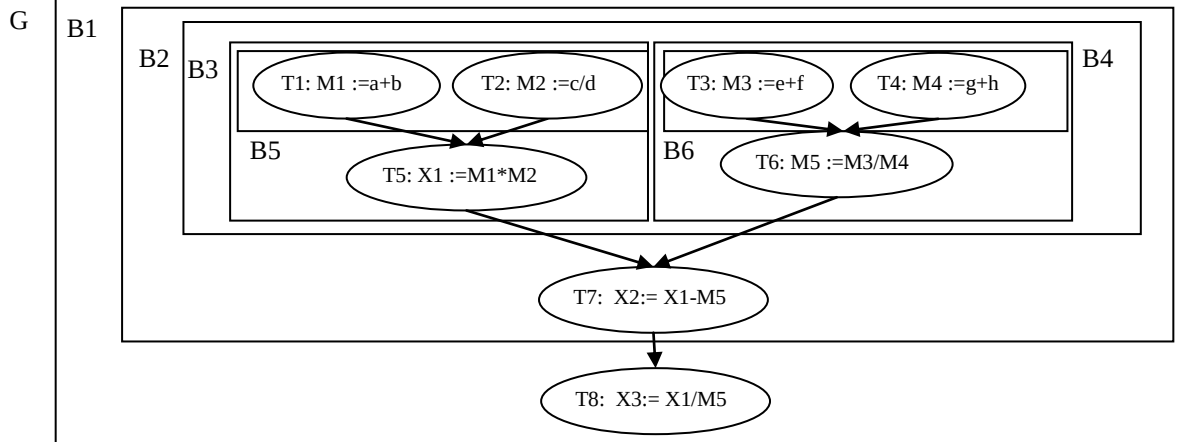
Exercice n°1 : (6 pts= 2.5 + 1+ 2.5)

A/ Le graphe de précédences de parallélisme maximal.



Un arc existe entre T6 et T8, mais il est supprimé car il est redondant.

B/ Le graphe est proprement lié car il est complètement décomposable de manière hiérarchique en tâches, soit parfaitement séquentielles, soit parfaitement parallèles, comme suit:



$G \equiv B1 \rightarrow T8.$
 $B1 \equiv B2 \rightarrow T7$
 $B2 \equiv B3 \parallel B4$
 $B3 \equiv B5 \rightarrow T5$
 $B4 \equiv B6 \rightarrow T6$
 $B5 \equiv T1 \parallel T2$
 $B6 \equiv T3 \parallel T4.$

C/ Le programme à l'aide de *Pabegin Parend*.

```

Debut
Parbegin
Debut
Parbegin T1 ; T2 Parend ;
T5
Fin ;
Debut
Parbegin T3 ; T4 Parend ;
T6
Fin
Parend ;
T7 ;
T8
Fin.
  
```

Exercice n°2 : (7 pts=3,5+1.5+2)

A/ - Cette solution met en œuvre le modèle du producteur / consommateur dans laquelle un producteur P cyclique dépose à chaque itération une copie d'un même message produit dans 2 tampons $T1$ et $T2$ de tailles respectives de N cases qu'utilisent respectivement les processus consommateurs $P1$ et $P2$ cycliques. Les deux familles de processus accèdent aux tampons de manière alternée. - **1 pt**-

- Le producteur, lors de son cycle, produit tout d'abord un article puis ne le dépose dans chacun des deux tampons que s'il y a au moins une case vide dans chacun de ces deux tampons ($P(nv1)$; $P(nv2)$). Puisque $nv1$ et $nv2$ sont initialisées à 1, cela signifie que P ne peut déposer un nouveau message que si les deux processus $P1$ et $P2$ aient déjà vidés les messages déposés la dernière fois par P . Après le dépôt, il déclare une case pleine dans chacun des deux tampons ($V(np1)$; $V(np2)$) et ainsi $P1$ et $P2$ peuvent accéder au tampon pour vider la case. Par conséquent, chaque consommateur Pi , $i=1$ ou 2, lors de son cycle, passe par $P(npi)$, $i=1$ ou 2 pour s'assurer de l'existence d'une case pleine dans le tampon Ti puis prélève une case pleine en la déclarant ensuite comme case vide ($V(nvi)$), $i=1$ ou 2 puis consomme le message prélevé et ainsi le cycle se répète.

Si aucune case n'est pleine (respectivement vide), le processus correspondant se bloque jusqu'à ce que le processus correspondant remplisse (respectivement vide) une case. En résumé, au maximum une case peut être pleine dans chacun des deux tampons à tout instant. Ainsi l'accès au tampon est alterné : P ; $P1$; $P2$ (ou $P2$; $P1$) ; P , $P1$; $P2$ - **2,5 pt**-

B/ L'écart maximal entre $np1$ et $np2$ est $1+1=2$, c'est le cas où, par exemple, P a rempli une case dans chaque tampon ; puis $P1$ a vidé la seule case remplie par P et revient pour se bloquer ($np1=-1$) et $P2$ n'a pas encore vidé la seule case remplie par P ($np2=1$).

C/ - L'inconvénient principal est le fait que le producteur ne peut déposer un prochain article dans les deux tampons que si $P1$ et $P2$ aient vidés le dernier article déposé par P . Si un consommateur est lent (ne vide pas son tampon) relativement à l'autre, le producteur ne peut pas progresser. Par conséquent, le producteur fonctionne au rythme du plus lent consommateur, ce qui retarde le consommateur le plus rapide.

- De plus, il y a une sous-utilisation des deux tampons car au maximum une case peut se trouver remplie à tout instant.

Exercice n°3 : (7 pts=1+1+5)

A/ La forme générale d'un processus:

Processus $P(i : \text{entier})$;

Début

-
 $M.\text{demander}(k)$ // M : est l'identifiant du moniteur
 < Utiliser les ressources >
 $M.\text{libérer}(k)$

-

Fin.

Le processus peut libérer par parties les ressources préalablement acquises.

B/ Les structures principales

- f : une liste explicite dont chaque élément est un enregistrement de structure id : identité du processus ayant une demande pendante et nb : le nombre d'exemplaires demandés. Cette liste est triée par ordre croissant du nombre d'instances demandées. A cette liste, sont associées les primitives suivantes :
 - $\text{insérer}(f, i, k)$ insère un élément contenant i et k dans la liste en conservant l'ordre croissant des valeurs de k .
 - $\text{extraire}(f)$: supprime le premier élément de la liste.
 - $\text{vide}(f)$: retourne vrai si la liste est vide.
 - $\text{premier}(f)$: retourne le premier élément de la liste.
- c : un tableau de conditions de taille N dont $c[i]$ est la condition sur laquelle Pi se bloque.
- Dispo : indique le nombre d'instances disponibles de la ressource à tout instant.

- **Remarque : On peut utiliser à la place de la file f un tableau d'entiers de taille N pour conserver le nombre de demandes de chaque processus bloqué, mais il doit être géré convenablement.**

C/ La solution :

M : Moniteur ;

Const $N = \dots$; $M = \dots$;

Var c : Tableau $[1..N]$ de condition ;

Dispo : entier ;

<déclaration de la file f + ses procédures d'accès>

- 0,75 pt-

Entry procedure demander (i : entier, k : entier) ;

Debut

Si (Dispo < k) Alors insérer (f , i , k) ; $c[i].wait()$ Fsi ;

Dispo := Dispo - k ;

Fin ;

- 1 pt-

Entry procedure libérer (k : entier) ;

Debut

Dispo := Dispo + k ;

Tant que non vide(f) et (Dispo ≤ premier(f).nb) Faire

Dispo := Dispo - premier(f).nb ; extraire(f) ;

$c[\text{premier}(f).\text{id}].\text{signal}()$;

Fait

- 3 pt-

Fin ;

Initialisation

Debut

Dispo := M ;

Fin.

- 0,25 pt-